1. What is the purpose of a loop structure?

The purpose of a loop structure is to repeat a set of instructions while a certain condition is true. Loops are useful for tasks such as adding numbers, repeating calculations, or asking a user to enter valid information.

2. Explain the difference between a while statement and a do-while statement.

A `while` statement checks its condition before running the code, so it might not run at all if the condition is false at the beginning. A `do-while` statement checks its condition after running the code, which means it always runs at least once.

3. An input validation loop is a loop that checks user input for valid data. If valid data is not entered, the loop iterates until valid data is entered. In which review of this chapter did you write code for an input validation loop?

The input validation loop was used in the Review: WordGuess section of the chapter, where the program repeatedly asks the player for input until the game reaches a stopping condition. The chapter also demonstrates input validation with a `do-while` loop that asks the user to enter a number less than 4.

lvp_java_text.pdf

4. a) What is an infinite loop?

An infinite loop is a loop that continues running forever because its stopping condition never becomes false.

b) List two types of errors that can lead to an infinite loop.

Two types of errors that can cause an infinite loop are syntax errors and logic errors.

c) What is meant by overflow?

Overflow happens when a number becomes too large or too small to be stored in the available number of bits. This can cause the stored value to change unexpectedly.

5. How many times will the do-while loop execute?

```
int x = 0;
do {
   x = x + 2;
} while (x < 120);
```

The loop will execute 60 times. The value of x increases by 2 each time, starting at 0, until it reaches 120. At that point, the condition becomes false and the loop stops.

6. What initial value of x would make the loop infinite?

```
do {
   x = x + 3;
} while (x < 120);
```

As written, there is no initial integer value of x that makes this loop infinite in Java. The loop increases x by 3 and eventually reaches or passes 120, making the condition false. Even if integer overflow occurs, the value will eventually reach the stopping condition. The question may contain a typo, so it would be worth checking the original textbook or asking your teacher.

7. Compare and contrast counters and accumulators. List two uses for each.

A counter keeps track of how many times something happens and usually increases by 1 each time. An accumulator adds values together and can increase by different amounts.

Two uses for counters are:

- Keeping track of the number of guesses in a game.
- Counting how many values a user enters.

Two uses for accumulators are:

- Calculating the total of a group of numbers.
- Keeping a running total of points earned in a game.

8. Write a for statement that sums the integers from 3 to 10, inclusive.

```
int sum = 0;

for (int i = 3; i <= 10; i++) {
   sum += i;
}

System.out.println(sum);
```

The output is 52 because the program adds all the integers from 3 through 10.

9. List two factors that should be considered when determining which loop structure to choose.

Two factors to consider are:

- Whether the loop needs to execute at least once.
- Whether the number of repetitions is known in advance or depends on a condition being met.

10. a) List two methods for debugging an application.

Two methods for debugging an application are:

- Using additional `println()` statements to display variable values and check what the program is doing.
- Stepping through the code using a debugger to examine variables and find where an error occurs.

b) Which method listed in part (a) would be best for debugging an application that generates random numbers that are used for determining the condition of an if statement? Why?

Using additional `println()` statements would be a good method because they can display the random number generated and show which condition the program is checking. This makes it easier to understand why the `if` statement follows a particular path, especially when the random number changes each time the program runs.